

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю № 1

Вариант № 23

Вариант запросов В

Выполнил:
Студент группы ИУ5-34Б
Кострыкина Е. Г.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Задание рубежного контроля

Рубежный контроль представляет собой разработку программы на языке Python, которая выполняет следующие действия:

- 1) Необходимо создать два класса данных в соответствии с Вашим вариантом предметной области, которые связаны отношениями один-ко-многим и многие-ко-многим.
- 2) Пример классов данных для предметной области Сотрудник-Отдел:
 1. Класс «Сотрудник», содержащий поля:
 - ID записи о сотруднике;
 - Фамилия сотрудника;
 - Зарплата (количественный признак);
 - ID записи об отделе. (для реализации связи один-ко-многим)
 2. Класс «Отдел», содержащий поля:
 - ID записи об отделе;
 - Наименование отдела.
 3. (Для реализации связи многие-ко-многим) Класс «Сотрудники отдела», содержащий поля:
 - ID записи о сотруднике;
 - ID записи об отделе.

2) Необходимо создать списки объектов классов, содержащих тестовые данные (3-5 записей), таким образом, чтобы первичные и вторичные ключи соответствующих записей были связаны по идентификаторам.

3) Необходимо разработать запросы в соответствии с Вашим вариантом. Запросы сформулированы в терминах классов «Сотрудник» и «Отдел», которые используются в примере. Вам нужно перенести эти требования в Ваш вариант предметной области. При разработке запросов необходимо по возможности использовать функциональные возможности языка Python (list/dict comprehensions, функции высших порядков).

Для реализации запроса №2 введите в класс, находящийся на стороне связи «много», произвольный количественный признак, например, «зарплата сотрудника».

Результатом рубежного контроля является документ в формате PDF, который содержит текст программы и результаты ее выполнения.

Задание варианта

Классы: «Синтаксическая конструкция» и «Язык программирования»

Вариант В.

1. «Синтаксическая конструкция» и «Язык программирования» связаны соотношением один-ко-многим. Выведите список всех конструкций, у которых название начинается с буквы «А», и названия их языков программирования.

2. «Синтаксическая конструкция» и «Язык программирования» связаны соотношением один-ко-многим. Выведите список языков с минимальным временем компиляции конструкции в каждом языке программирования, отсортированный по минимальному времени компиляции.

3. «Синтаксическая конструкция» и «Язык программирования» связаны соотношением многие-ко-многим. Выведите список всех связанных конструкций и языков, отсортированный по конструкциям, сортировка по языкам программирования произвольная.

Код программы

```
from operator import itemgetter

class ProgrammingLanguage:
    """Язык программирования"""

    def __init__(self, pl_id, name):
        self.id = pl_id
        self.name = name

class SyntaxConstruction:
    """Синтаксическая конструкция"""

    def __init__(self, sc_id, name, compile_time, pl_id):
        self.id = sc_id
        self.name = name
        self.compile_time = compile_time    # Время компиляции в
        # миллисекундах - количественный признак
        self.pl_id = pl_id

class LanguageConstruction:
    """Конструкции языков для связи многие-ко-многим"""

    def __init__(self, sc_id, pl_id):
        self.sc_id = sc_id
        self.pl_id = pl_id

# Языки программирования
languages = [
    ProgrammingLanguage(1, "Python"),
    ProgrammingLanguage(2, "Java"),
    ProgrammingLanguage(3, "C++"),
    ProgrammingLanguage(4, "JavaScript")
]

# Синтаксические конструкции
constructions = [
    SyntaxConstruction(1, "Лямбда-функция", 3, 1),
    SyntaxConstruction(2, "Интерфейс", 8, 2),
    SyntaxConstruction(3, "Абстрактный класс", 10, 2),
    SyntaxConstruction(4, "Шаблон функции", 12, 3),
    SyntaxConstruction(5, "Асинхронная функция", 7, 4)
]

# Связи многие-ко-многим
language_constructions = [
    LanguageConstruction(1, 1),
    LanguageConstruction(2, 2),
    LanguageConstruction(3, 2),
    LanguageConstruction(4, 3),
    LanguageConstruction(5, 1),
]

def main():
    """Основная функция"""

    # Соединение данных один-ко-многим
```

```

one_to_many = [(sc.name, sc.compile_time, pl.name)
               for pl in languages
               for sc in constructions
               if sc.pl_id == pl.id]

# Соединение данных многие-ко-многим
many_to_many_temp = [(pl.name, lc.pl_id, lc.sc_id)
                     for pl in languages
                     for lc in language_constructions
                     if pl.id == lc.pl_id]

many_to_many = [(sc.name, sc.compile_time, pl.name)
                for pl_name, pl_id, sc_id in many_to_many_temp
                for sc in constructions if sc.id == sc_id]

print('\nЗадание B1')
# Список синтаксических конструкций, название которых начинается с 'A',
# и названия языков, в которых они содержатся:
result_1 = list(filter(lambda i: i[0].startswith('A'), one_to_many))
# Сортируем по названию конструкции
result_1_sorted = sorted(result_1, key=itemgetter(0))

for construction_name, compile_time, language_name in result_1_sorted:
    print(f"Конструкция: {construction_name:<25} Язык: {language_name}")

print('\nЗадание B2')
# Список языков с минимальным временем компиляции конструкций в каждом
языке программирования,
# отсортированный по минимальному времени компиляции
result_2_unsorted = []
# Перебираем все языки программирования
for pl in languages:
    # Список конструкций языка
    l_constructions = list(filter(lambda i: i[2] == pl.name,
one_to_many))
    # Если конструкция не пустая
    if len(l_constructions) > 0:
        # Времена компиляции конструкций языка
        l_times = [time for _, time, _ in l_constructions]
        # Минимальное время компиляции
        l_min_time = min(l_times)
        result_2_unsorted.append((pl.name, l_min_time))

# Сортировка по минимальному времени компиляции
result_2 = sorted(result_2_unsorted, key=itemgetter(1))

for language_name, min_time in result_2:
    print(f"Язык: {language_name:<15} Мин. время компиляции: {min_time}
мс")

print('\nЗадание B3')
# Список всех связанных конструкций и языков, отсортированный по
конструкциям
result_3 = sorted(many_to_many, key=itemgetter(0))

for construction_name, compile_time, language_name in result_3:
    print(f"Конструкция: {construction_name:<25} Язык: {language_name}")

if __name__ == "__main__":
    main()

```

Результаты выполнения

Задание B1

Конструкция: Абстрактный класс

Язык: Java

Конструкция: Асинхронная функция

Язык: JavaScript

Задание B2

Язык: Python

Мин. время компиляции: 3 мс

Язык: JavaScript

Мин. время компиляции: 7 мс

Язык: Java

Мин. время компиляции: 8 мс

Язык: C++

Мин. время компиляции: 12 мс

Задание B3

Конструкция: Абстрактный класс

Язык: Java

Конструкция: Асинхронная функция

Язык: Python

Конструкция: Интерфейс

Язык: Java

Конструкция: Лямбда-функция

Язык: Python

Конструкция: Шаблон функции

Язык: C++